CONTROLE DE TRANSAÇÃO

INTRODUÇÃO

- Uma transação é uma série de operações sequenciais realizadas no Banco de Dados.

- As operações da transação compõem uma unidade de trabalho, ou seja, é como se fossem uma coisa só.

- Existem 4 propriedades das transações ACID: atomicidade, consistência, isolamento e durabilidade

PROPRIEDADES DAS TRANSAÇÕES

- A atomicidade trata as transações como um conjunto, de forma atômica. Em outras palavras, ou a transação será executada totalmente ou não será executada. “É o tudo ou nada”

- A consistência prega que o Banco de Dados saia de um estado consistente para outro também consistente, após a realização de uma transação.

PROPRIEDADES DAS TRANSAÇÕES

- Imagine que precisamos realizar em uma única transação duas operações de manipulação de dados (DML, do inglês Data Manipulation Language), como INSERT e UPDATE.

- Estas operações só podem se tornar permanentes no banco de dados se todas forem executadas com sucesso.

- Em caso de falhas em uma das duas é possível cancelar a transação, porém todas modificações realizadas durante a mesma serão descartadas, inclusive a que obtive sucesso.

- Assim, os dados permanecem íntegros e consistentes.

PROPRIEDADES DAS TRANSAÇÕES

- O isolamento trata as transações de forma independente, isoladas umas das outras. A ideia é que o funcionamento de uma transação não interfira na outra, visando minimizar conflitos por concorrências.

- A durabilidade é a persistência das transações no Banco de Dados, após a sua efetivação. Em outras palavras, uma vez que as transações foram confirmadas, elas não podem ser desfeitas por erros futuros.

BEGIN COMMIT ROLLBAK

```
BEGIN;

CREATE TABLE cliente (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100)
);

CREATE TABLE pedido (
    id SERIAL PRIMARY KEY,
    cliente_id INT REFERENCES cliente(id),
    data_pedido DATE DEFAULT CURRENT_DATE
);

COMMIT;
```

FUNÇÕES E PROCEDIMENTOS

INTRODUÇÃO

- **Funções** e **Procedimentos** são ambos blocos de código armazenados que podem ser executados no servidor do banco de dados

- servem para automatizar cálculos, regras de negócio ou transformações de dados, etc

- mas possuem diferenças fundamentais.

FUNÇÕES

- Podem retornar um valor (ou conjunto de valores).
- Podem ser usadas em consultas SQL (SELECT, WHERE, etc.).
- Executadas dentro de uma única transação.
- Não podem executar comandos de controle de transação (COMMIT, ROLLBACK).

FUN

Você pode chamar uma função diretamente dentro de um SELECT, como se fosse um campo para retorno, ou usá-la para filtrar dados no WHERE. Isso permite reaproveitar regras e cálculos com facilidade.

- Podem ser usadas em consultas SQL (SELECT, WHERE, etc.).
- Executadas dentro de uma única transação.
- Não podem executar comandos de controle de transação (COMMIT, ROLLBACK).

FUNÇÕES

Uma unidade lógica de trabalho que contém uma ou mais **operações** (como *inserções, atualizações ou exclusões*) que devem ser executadas de forma atômica - ou seja, ou todas as operações são realizadas com sucesso, ou nenhuma delas é efetivada.

Se der erro no meio, ela não deixa alterações parciais no banco.

- Executadas dentro de uma única **transação**.

- Não podem executar comandos de controle de transação (COMMIT, ROLLBACK).

FUN

Dentro de uma função não é permitido fazer COMMIT ou ROLLBACK, porque isso quebraria justamente a regra anterior. Quem controla a transação é quem chamou a função, e não a função em si.

COMMIT:

- O COMMIT é o comando para confirmar/efetivar todas as alterações feitas na transação no banco de dados.

ROLLBACK:

- O ROLLBACK é o comando para desfazer/reverter todas as alterações feitas na transação no banco de dados.

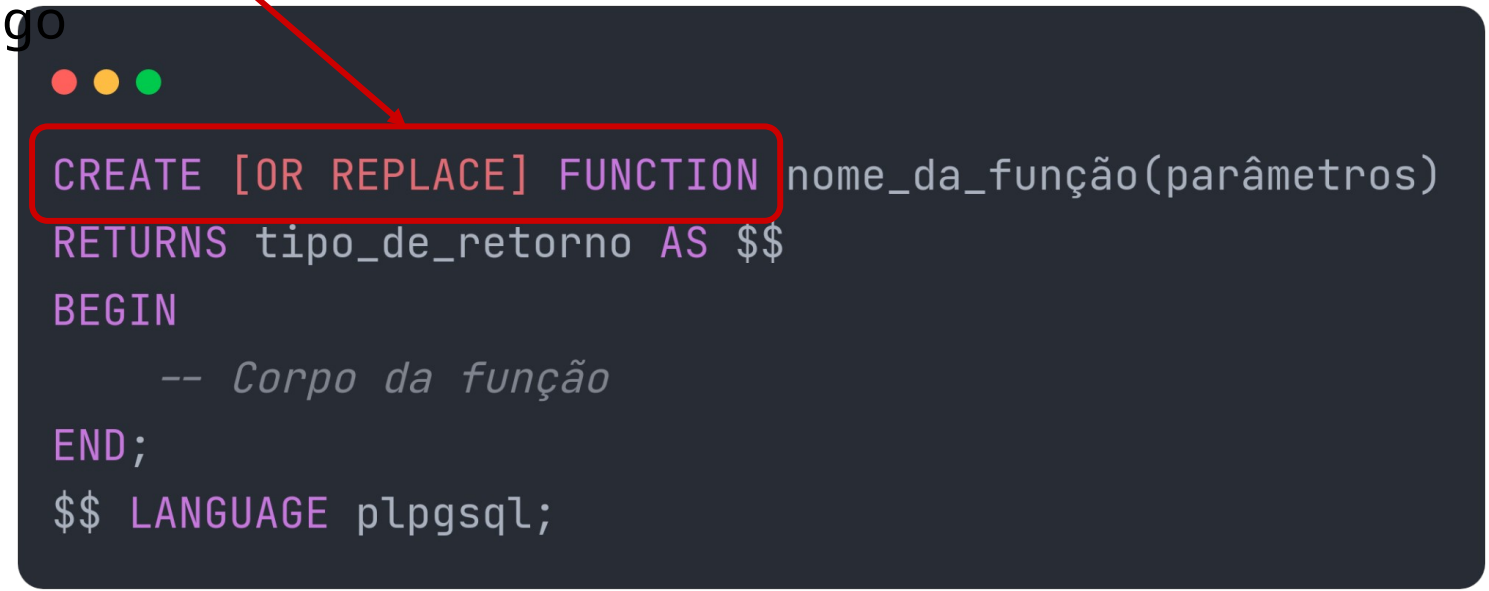
- Não pode executar comandos de controle de transação (**COMMIT**, **ROLLBACK**).

SINTAXE

- Podem ser escritas em várias linguagens, sendo a mais comum a PL/pgSQL (Procedural Language/PostgreSQL).

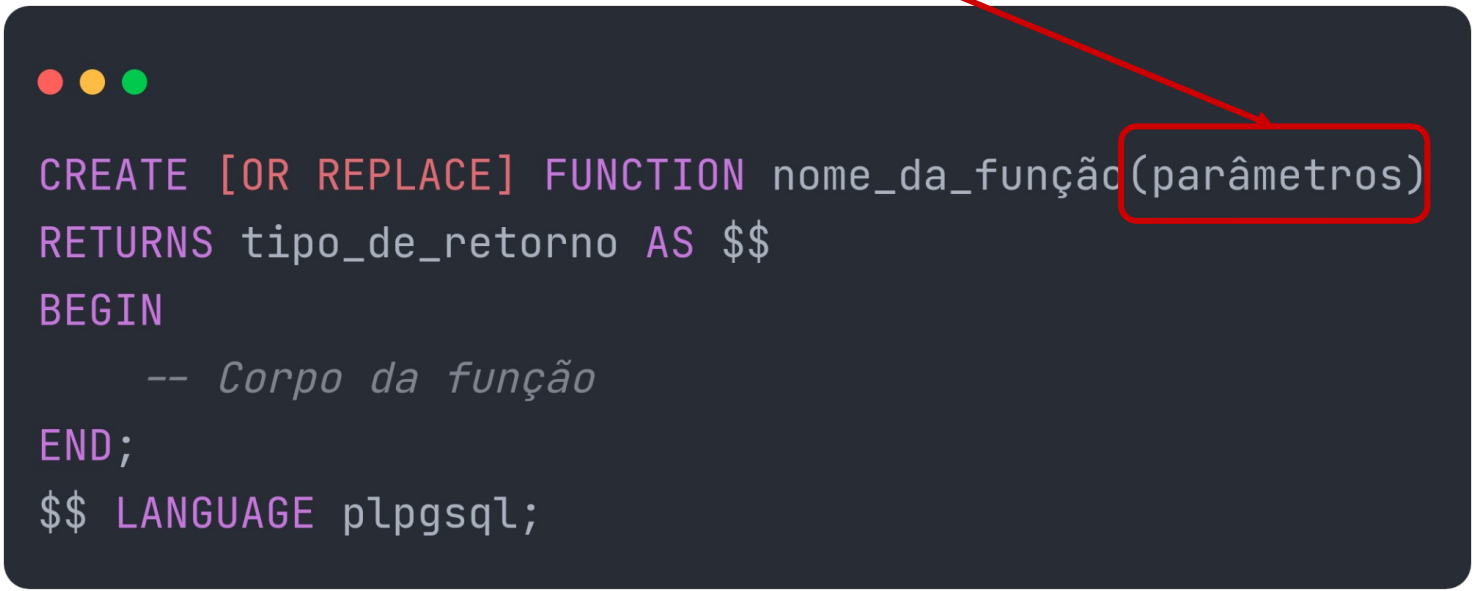
```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Cria uma nova função, se não existir, ou substitui uma existente pelo novo código



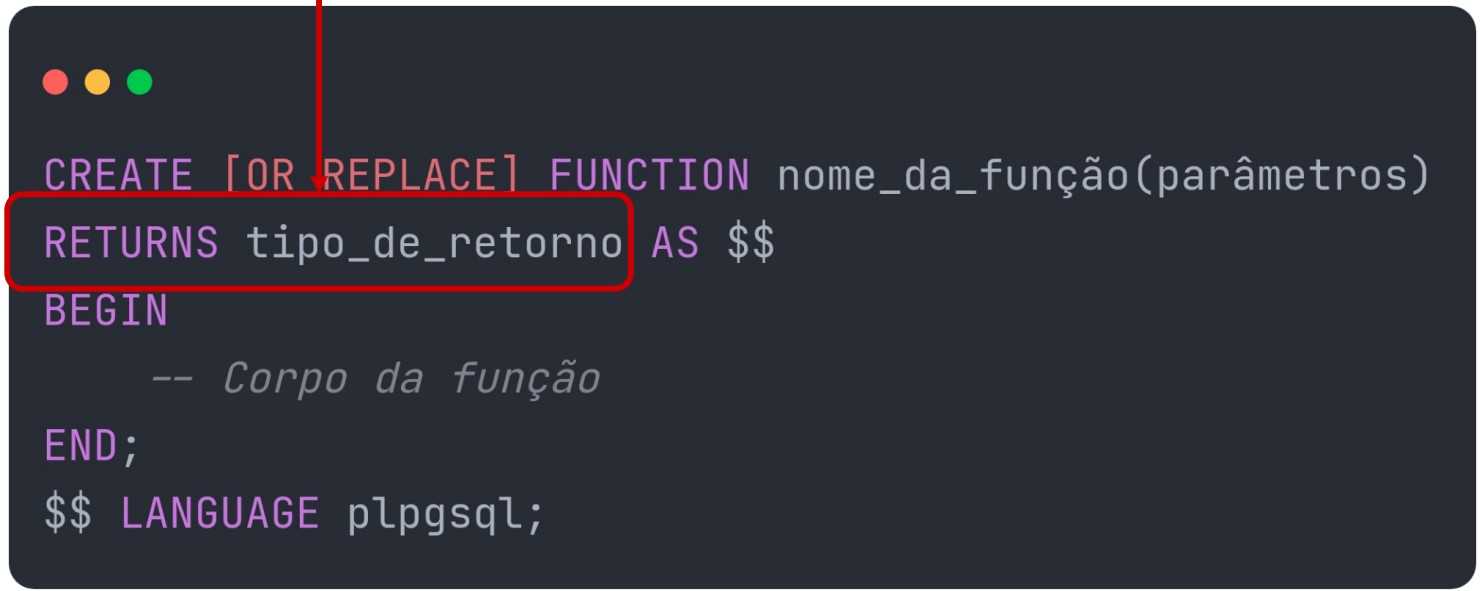
```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Lista de parâmetros de entrada



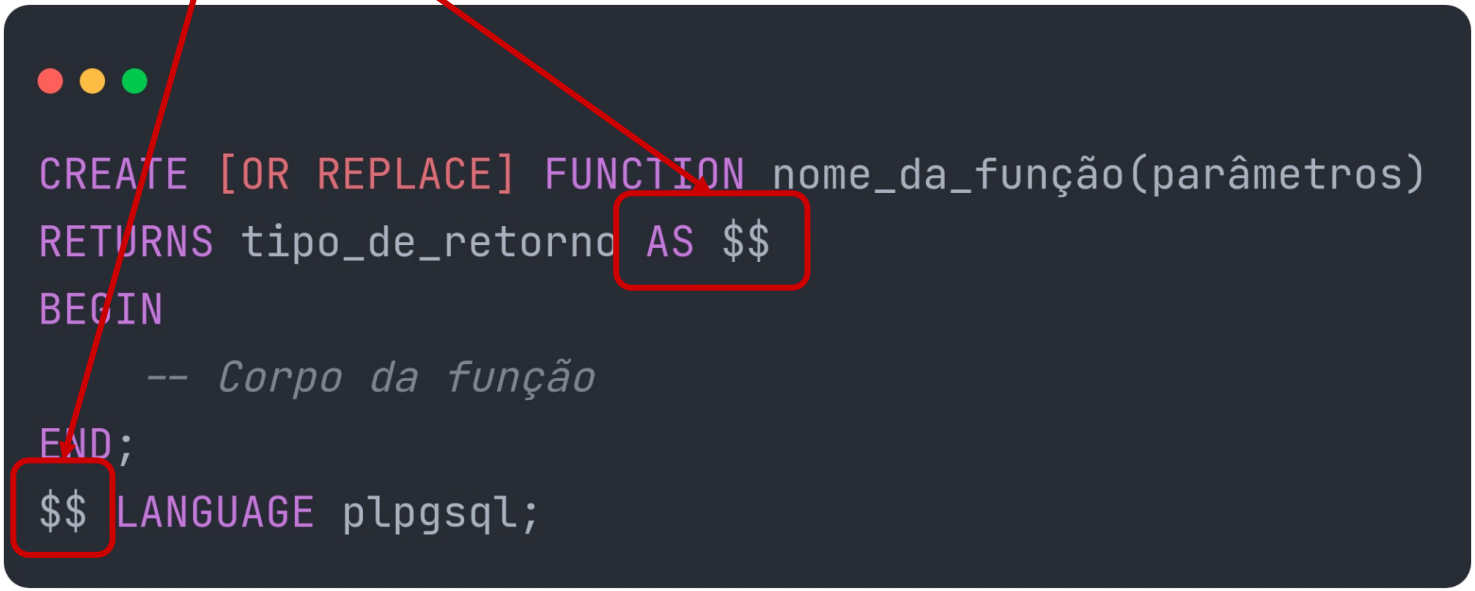
```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Define o tipo de retorno da função.

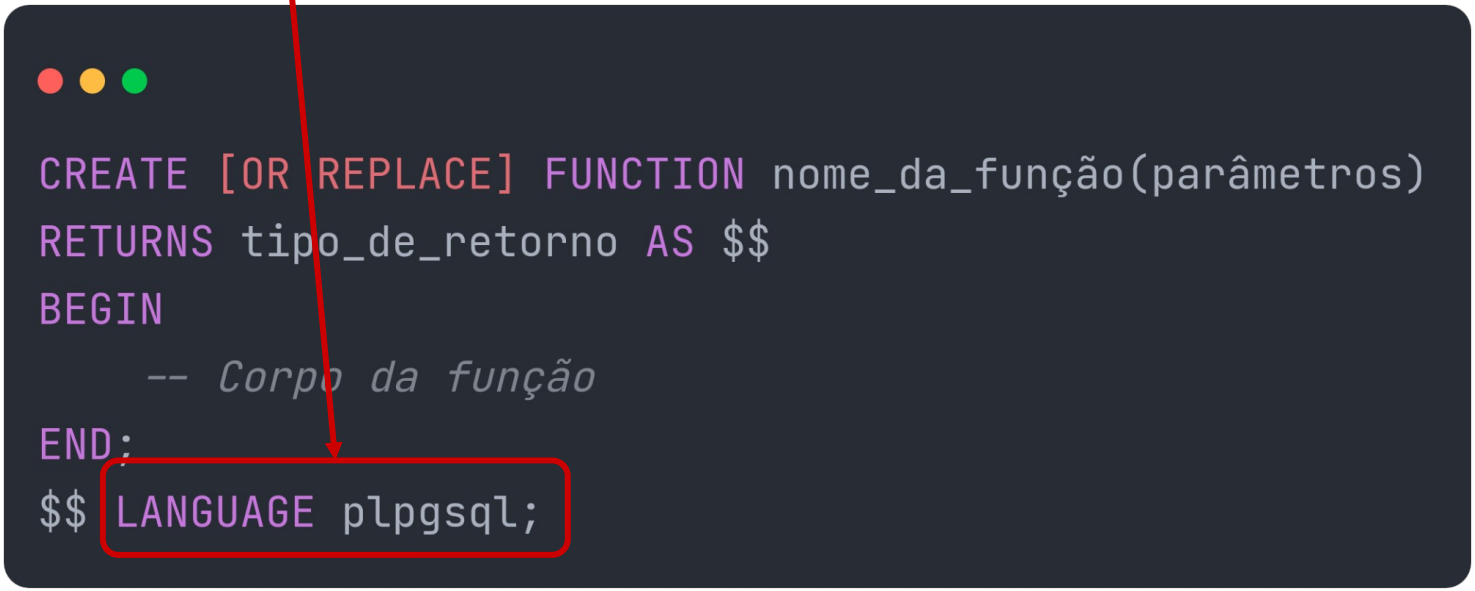


```
CREATE [OR REPLACE] FUNCTION nome_da_função(parâmetros)
RETURNS tipo_de_retorno AS $$
BEGIN
    -- Corpo da função
END;
$$ LANGUAGE plpgsql;
```

Delimitador do corpo da função.



Especifica a linguagem utilizada.



EXEMPLO (1) - FUNÇÕES

Criar uma função que recebe o **ID de um cliente** e retorna o ***total gasto por esse cliente em todos os seus pedidos***.

```sql
CREATE OR REPLACE FUNCTION calcular_total_cliente(cliente_id_input INTEGER)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    total_cliente NUMERIC(10, 2);
BEGIN
    SELECT SUM(total) INTO total_cliente
    FROM pedidos
    WHERE cliente_id = cliente_id_input;

    -- Se o cliente não tiver pedidos, retornar zero
    IF total_cliente IS NULL THEN
        total_cliente := 0;
    END IF;

    RETURN total_cliente;
END;
$$ LANGUAGE plpgsql;
```

```sql
CREATE OR REPLACE FUNCTION calcular_total_cliente(cliente_id_input INTEGER)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    total_cliente NUMERIC(10, 2);
BEGIN
    SELECT SUM(total) INTO total_cliente
    FROM pedidos
    WHERE cliente_id = cliente_id_input;

    -- Se o cliente não tiver pedidos, retornar zero
    IF total_cliente IS NULL THEN
        total_cliente := 0;
    END IF;

    RETURN total_cliente;
END;
$$ LANGUAGE plpgsql;
```

Declaramos uma variável local

Armazena o resultado da soma os valores da coluna total para aquele cliente específico dentro da variável total_cliente

```sql
CREATE OR REPLACE FUNCTION calcular_total_cliente(cliente_id_input INTEGER)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    total_cliente NUMERIC(10, 2);
BEGIN
    SELECT SUM(total) INTO total_cliente
    FROM pedidos
    WHERE cliente_id = cliente_id_input;

    -- Se o cliente não tiver pedidos, retornar zero
    IF total_cliente IS NULL THEN
        total_cliente := 0;
    END IF;

    RETURN total_cliente;
END;
$$ LANGUAGE plpgsql;
```

IF condição THEN
-- comandos
END IF;

:= é o operador de atribuição

- Utilizando a função

```
SELECT nome, calcular_total_cliente(cliente_id) AS total_gasto
FROM clientes;
```

- Resultado

	nome	total_gasto
1	Theresa Collins	150.00
2	Joshua Tucker	0.00
3	James Smith	0.00

EXEMPLO (2) - FUNÇÕES

Criar uma função que retorna o estoque disponível de um produto específico.

```sql
CREATE OR REPLACE FUNCTION obter_estoque_disponivel(produto_id_input INTEGER)
RETURNS INTEGER AS $$
DECLARE
    estoque_disponivel INTEGER;
BEGIN
    SELECT disponivel INTO estoque_disponivel
    FROM produtos
    WHERE produto_id = produto_id_input;

    RETURN estoque_disponivel;
END;
$$ LANGUAGE plpgsql;
```

- Utilizando a função

```sql
SELECT nome, obter_estoque_disponivel(produto_id) AS estoque
FROM produtos
WHERE produto_id = 1;
```

- Resultado

	nome	estoque
1	Notebook Dell Inspiron	10

EXEMPLO (3) - FUNÇÕES

Função que retorna todos os produtos pertencentes a uma categoria específica.

```sql
CREATE OR REPLACE FUNCTION listar_produtos_por_categoria(categoria_id_input INTEGER)
RETURNS TABLE(produto_id INTEGER, nome VARCHAR, preco NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT p.produto_id, p.nome, p.preco
    FROM produtos p
    JOIN produtos_categorias pc ON p.produto_id = pc.produto_id
    WHERE pc.categoria_id = categoria_id_input;
END;
$$ LANGUAGE plpgsql;
```

- Utilizando a função

```
SELECT * FROM listar_produtos_por_categoria(14);
```

- Resultado

	produto_id	nome	preco
1	31	Calça Legging Fitness	90.00
2	4	Vestido Longo Floral	120.00

IF / ELSE IF / ELSE

```
CREATE OR REPLACE FUNCTION verificar_idade(idade INTEGER) RETURNS TEXT AS $$
BEGIN
    IF idade < 18 THEN
        RETURN 'Menor de idade';
    ELSIF idade ⩾ 18 AND idade < 65 THEN
        RETURN 'Adulto';
    ELSE
        RETURN 'Idoso';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

e muito mais...

- LOOP - continua executando até encontrar um EXIT.

```
LOOP
    -- comandos
    EXIT WHEN condição;  -- encerra o loop quando a condição for verdadeira
END LOOP;
```

- WHILE - enquanto uma condição é verdadeira.

```
WHILE condição LOOP
    -- comandos
END LOOP;
```

e muito mais...

- CASE

```
CASE
    WHEN condição1 THEN
        -- comandos
    WHEN condição2 THEN
        -- comandos
    ELSE
        -- comandos padrão
END CASE;
```

e muito mais...

- FOR ... IN (para iterações sobre valores) ou sobre resultados de uma consulta.

```
FOR i IN 1..10 LOOP
    -- comandos usando i
END LOOP;
```

```
FOR registro IN SELECT id, nome FROM clientes LOOP
    -- usar registro.id ou registro.nome
END LOOP;
```

PROCEDIMENTOS

- Introduzidos no PostgreSQL 11.
- Não retornam valores diretamente (podem usar parâmetros OUT).
- Executados usando o comando CALL.
- Não podem ser usadas em consultas SQL.
- Podem executar comandos de controle de transação.

PRO

Diferente das funções, os procedimentos não usam RETURN valor.
Para devolver dados, precisam usar parâmetros de saída (OUT) ou registrar resultados em tabelas.

- Não retornam valores diretamente (podem usar parâmetros OUT).
- Executados usando o comando CALL.
- Não podem ser usadas em consultas SQL.
- Podem executar comandos de controle de transação.

PROCEDIMENTOS

Enquanto funções são chamadas com SELECT nome_funcao(), os procedimentos são executados com:

CALL nome_procedimento(param1, param2);

Procedimentos só podem ser chamados com CALL, nunca em um SELECT.

- Executados usando o comando CALL.
- Não podem ser usadas em consultas SQL.
- Podem executar comandos de controle de transação.

PROCEDIMENTOS

- Não podem ser usadas em consul

Essa é a maior diferença em relação às funções.

Dentro de um procedimento, você pode usar COMMIT, ROLLBACK,

Por esse motivo, se você precisar de um bloco de lógica que atualiza dados e controla transações diretamente, o ideal é usar um PROCEDIMENTO

- Podem executar comandos de controle de transação.

CRIANDO PROCEDIMENTOS

SINTAXE

- Procedimentos permitem executar blocos de código armazenados no banco de dados. Eles são úteis para encapsular operações que não precisam retornar valores diretamente.

```
CREATE [OR REPLACE] PROCEDURE nome_do_procedimento(parâmetros)
LANGUAGE plpgsql AS $$
BEGIN
    -- Corpo do procedimento
END;
$$;
```

EXEMPLO (1) - PROCEDIMENTOS

Procedimento que atualiza o estoque de um produto após a venda.

```sql
CREATE OR REPLACE PROCEDURE atualizar_estoque_produto(
    produto_id_input INTEGER,
    quantidade_vendida INTEGER
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE produtos
    SET
        estoque = estoque - quantidade_vendida,
        disponivel = disponivel - quantidade_vendida
    WHERE produto_id = produto_id_input;

    -- Verificar se o estoque ficou negativo
    IF (SELECT estoque FROM produtos WHERE produto_id = produto_id_input) < 0 THEN
        RAISE NOTICE 'Estoque insuficiente para o produto ID %.', produto_id_input;
    ELSE
        RAISE NOTICE 'Estoque atualizado com sucesso para o produto ID %.', produto_id_input;
    END IF;
END;
$$;
```

- Utilizando o procedimento

```
CALL atualizar_estoque_produto(1, 5);
```

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software